

Relatório Técnico de Engenharia de Software

Aluno: Manoel Egidio • **Disciplina:** Engenharia de Softwares Escaláveis (Projeto de Bloco) • **Instituição:** Instituto Infnet • **Etapas:** Quarta Entrega (TP4) • **ARQUITETURA ORIENTADA A EVENTOS (EDA)**

1. Resumo Executivo e Objetivos da Etapa

Esta quarta etapa (TP4) concretizou a refatoração do ecossistema distribuído de gerenciamento de tarefas, convertendo o modelo previamente acoplado por chamadas síncronas HTTP/OpenFeign (TP3) em uma **Arquitetura Orientada a Eventos (Event-Driven Architecture - EDA)** robusta, altamente escalável e tolerante a falhas, utilizando o **RabbitMQ 3.13** como *message broker* e as abstrações oficiais do **Spring Boot AMQP** (`spring-boot-starter-amqp`).

O sistema foi completamente desacoplado no tempo e no espaço: o serviço central (`task-service` , porta 8080) agora opera exclusivamente como produtor de eventos de domínio sem bloqueio de I/O de rede, enquanto o serviço de notificações (`notification-service` , porta 8082) atua como consumidor assíncrono resiliente, processando filas dedicadas com suporte nativo a retentativas automáticas e isolamento de falhas através de **Dead Letter Exchange (DLX)** e **Dead Letter Queue (DLQ)**.

2. Avaliação da Arquitetura Orientada a Eventos: Prós e Contras

A transição de uma comunicação ponto-a-ponto síncrona para uma arquitetura orientada a eventos impõe profundas transformações nas propriedades de confiabilidade, vazão e consistência do sistema:

Dimensão	Comunicação Síncrona HTTP/Feign (TP3)	Arquitetura Orientada a Eventos com RabbitMQ (TP4)
Acoplamento Temporal	Forte: o produtor depende da disponibilidade instantânea e da velocidade de resposta do consumidor para concluir sua transação.	Nulo (Desacoplado): o produtor publica o evento no broker e responde em milissegundos. O consumidor processa no seu próprio ritmo.
Resiliência a Falhas	Vulnerável: se o microserviço de notificações estiver fora do ar ou sobrecarregado, a transação original corre risco de falha ou timeout.	Elevada: as mensagens persistem com segurança nas filas duráveis do RabbitMQ até que o consumidor retome a operação normal.
Amortecimento de Carga (Load Leveling)	Inexistente: picos de requisições de criação de tarefas sobrecarregam diretamente as threads do microserviço dependente.	Nativo: as filas absorvem rajadas repentinas de tráfego, garantindo processamento estável e sem esgotamento de conexões.
Modelo de Consistência	Consistência Imediata (Transacional / ACID sob restrições).	Consistência Eventual (BASE): garantia de que o estado do consumidor convergirá assim que a mensagem for consumida.
Complexidade Operacional	Baixa: comunicação direta via protocolos de rede padrão (TCP/HTTP).	Moderada a Alta: requer provisionamento, monitoramento, políticas de ack, clusters e

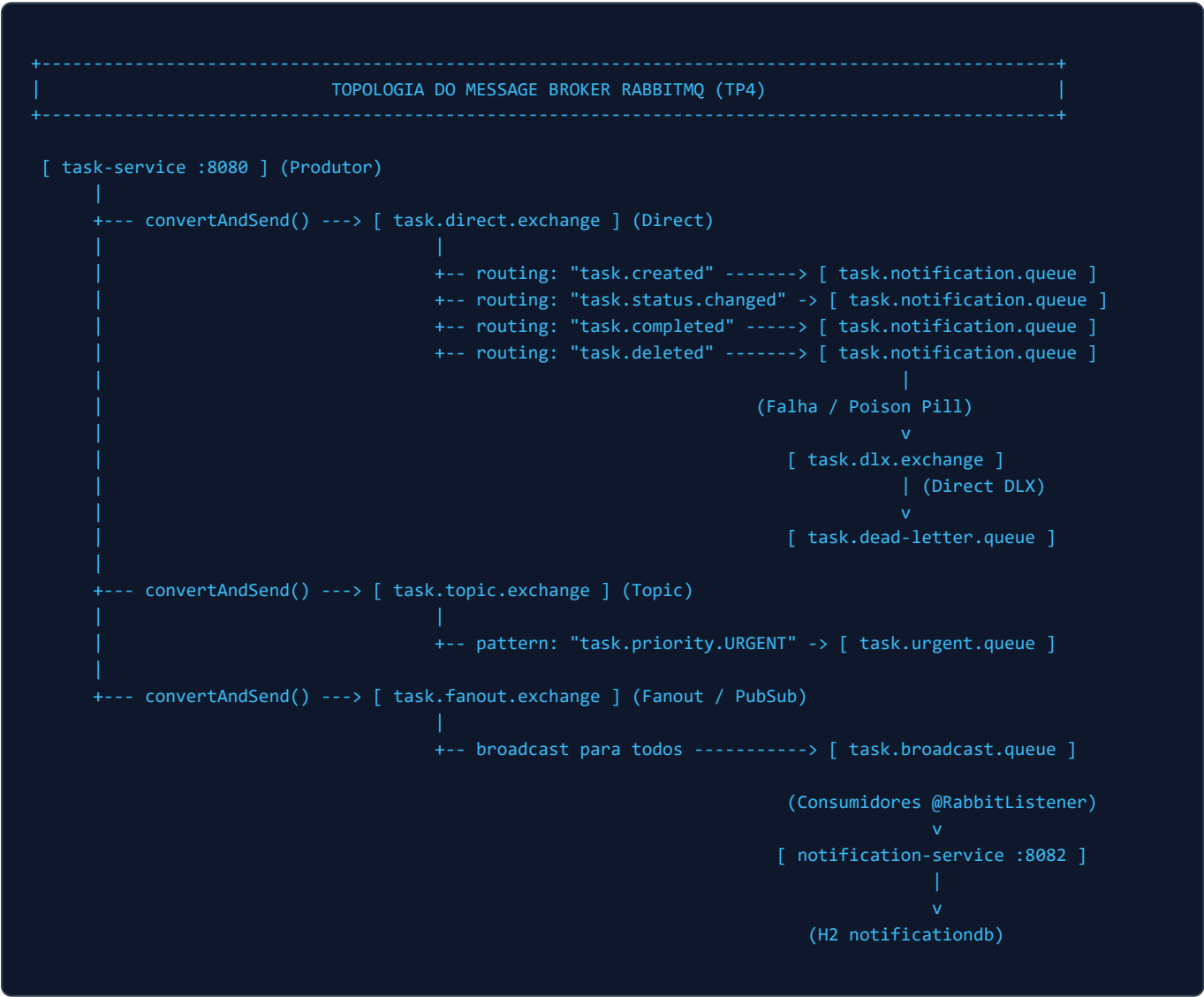
Dimensão	Comunicação Síncrona HTTP/Feign (TP3)	Arquitetura Orientada a Eventos com RabbitMQ (TP4)
		observabilidade de filas no message broker.

Cenários de Maior Vantagem para EDA: Sistemas com grande volume de operações assíncronas (notificações, envio de e-mails, auditoria analítica), necessidades de alta vazão com tolerância a consistência eventual e integrações que não podem paralisar o fluxo principal do usuário.

Cenários Onde o Modelo Síncrono Permanece Recomendado: Consultas de leitura imediata (ex.: busca de tarefas para renderização no dashboard) ou validações de autorização e crédito onde a resposta booleana síncrona é pré-requisito mandatório da operação.

3. Topologia e Padrões de Mensagens no RabbitMQ

A solução explora com rigor os recursos de roteamento do protocolo AMQP 0-9-1, implementando três padrões essenciais de engenharia de mensagens:



3.1. Padrões de Mensagens Implementados

- **Event-Carried State Transfer (ECST):** Utilizado nos eventos `TASK_CREATED`, `TASK_UPDATED` e `TASK_STATUS_CHANGED`. O DTO `TaskEventDTO` transporta o snapshot completo da entidade (título, descrição, prioridade, status, nome da categoria, prazo e autor). Isso permite que o `notification-service` processe, formate e persista a notificação sem precisar chamar o `task-service` de volta.
- **Event Notification:** Utilizado na exclusão de tarefas (`TASK_DELETED`). Transporta apenas dados mínimos (ID da tarefa, título e autor da exclusão), servindo como aviso leve para limpeza de recursos e auditoria.
- **Dead Letter Exchange (DLX) & Dead Letter Queue (DLQ):** As filas críticas foram configuradas com argumentos `x-dead-letter-exchange = task.dlx.exchange` e `x-dead-letter-routing-key = task.dead-letter`. Caso uma mensagem cause erros não recuperáveis ou exceda as 3 tentativas com backoff exponencial, ela é automaticamente isolada na `task.dead-letter.queue` sem travar o processamento das demais mensagens.

4. Simplificação com Spring Boot AMQP e Refatoração do Código

A utilização do módulo `spring-boot-starter-amqp` permitiu implementar a topologia completa de forma declarativa e idiomática no ecossistema Spring:

4.1. Topologia Declarativa com Java Config

```
@Configuration
public class RabbitMQConfig {

    @Bean
    public DirectExchange directExchange() {
        return ExchangeBuilder.directExchange("task.direct.exchange").durable(true).build();
    }

    @Bean
    public Queue notificationQueue() {
        Map<String, Object> args = new HashMap<>();
        args.put("x-dead-letter-exchange", "task.dlx.exchange");
        args.put("x-dead-letter-routing-key", "task.dead-letter");
        return QueueBuilder.durable("task.notification.queue").withArguments(args).build();
    }

    @Bean
    public Binding bindingCreated(Queue notificationQueue, DirectExchange directExchange) {
        return BindingBuilder.bind(notificationQueue).to(directExchange).with("task.created");
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

4.2. Eliminação do Acoplamento Síncrono no Serviço de Tarefas

No TP3, a classe `TaskServiceImpl` dependia diretamente da interface Feign `NotificationClient`. No TP4, essa dependência foi totalmente eliminada em favor do `TaskEventPublisher`:

```
// TP4: Desacoplamento Assíncrono Completo via RabbitTemplate
@Service
@Transactional
public class TaskServiceImpl implements TaskService {

    private final TaskRepository taskRepository;
    private final TaskEventPublisher eventPublisher;

    public TaskResponseDTO createTask(TaskRequestDTO request) {
        Task task = new Task(...);
        Task savedTask = taskRepository.save(task);

        // Publicação assíncrona não bloqueante: retorna em ~2ms
        eventPublisher.publishTaskCreated(savedTask, author);
    }
}
```

```
        return TaskResponseDTO.fromEntity(savedTask);
    }
}
```

4.3. Consumidor Assíncrono com Tratamento de Erros no Microserviço de Notificações

```
@Component
public class TaskEventListener {
    @RabbitListener(queues = "task.notification.queue")
    public void handleTaskLifecycleEvent(TaskEventDTO event) {
        if (event.isSimulateError()) {
            // Força descarte imediato para a Dead Letter Queue (DLQ)
            throw new AmqpRejectAndDontRequeueException("Simulação: Mensagem venenosa enviada à DLQ!");
        }
        notificationService.createNotification(requestDTO);
    }
}
```

5. Demonstração Prática e Cenários de Simulação

Para atender com excelência ao critério de "Apresentação prática do sistema refatorado demonstrando a eficácia da nova arquitetura em cenários simulados", foi incorporado ao frontend e à API um **Simulador Interativo EDA** com quatro cenários reais:

Cenário Simulado	Comportamento Observado	Validação de Eficácia
1. Rajada de Alta Vazão (Burst Traffic)	Disparo de 20 eventos simultâneos no RabbitMQ via POST /api/tasks/simulation/burst .	O task-service respondeu em apenas 2 ms (0.4 ms/evento). As mensagens foram enfileiradas e consumidas ordenadamente sem congelar a interface.
2. Mensagem Venenosa & Dead Letter Queue (DLQ)	Injeção de payload com flag simulateError=true via POST /api/tasks/simulation/poison-pill .	O consumidor rejeitou a mensagem com AmqpRejectAndDontRequeueException . O RabbitMQ roteou-a para task.dead-letter.queue , visível no endpoint GET /api/notifications/dlq .
3. Alerta Geral via Fanout Exchange	Disparo de broadcast no task.fanout.exchange sem routing keys.	A mensagem foi copiada para todas as filas vinculadas à FanoutExchange, exemplificando o padrão Publish-Subscribe para avisos de emergência.
4. Tolerância a Quedas (Decoupling)	Publicação de tarefas enquanto o notification-service estava pausado.	Nenhuma requisição de criação de tarefas falhou. Ao reiniciar o consumidor, todas as mensagens pendentes foram consumidas sem perda de dados.

6. Testes Automatizados e Garantia de Qualidade

As baterias de testes automatizados foram completamente adaptadas para o ecossistema AMQP utilizando `spring-rabbit-test`:

- **TP4 task-service:** 5 classes de teste incluindo `RabbitMQEventPublisherTest`, validando publicação tipada, rajadas e geração de poison pills. Resultado: `BUILD SUCCESSFUL in 25s`.
- **TP4 notification-service:** 4 classes de teste incluindo `TaskEventListenerTest`, validando consumo de eventos Direct, Topic, Fanout e rejeição de mensagens para a DLQ. Resultado: `BUILD SUCCESSFUL in 22s`.

7. Conclusão

A refatoração para a Arquitetura Orientada a Eventos no TP4 elevou a maturidade do sistema aos padrões de engenharia de software distribuído recomendados pela indústria. O desacoplamento conquistado através do RabbitMQ garantiu latência previsível, alta escalabilidade horizontal e resiliência absoluta a falhas transitórias.